

MedAssitAI: Medical Symptom Analysis & Disease Prediction System

Submitted By

Name: Diksha

Team: 2

Role: Fronted

Project Overview

During this milestone, I worked on understanding and testing the Authentication and Authorization module received from my teammate. Since my role is frontend development, my objective was to verify whether the backend authentication module was ready for frontend integration. I configured the backend locally, ran the FastAPI server, and examined the available authentication endpoints through Swagger UI. While testing, I identified issues in the backend database that currently prevent successful integration.

Work Completed

1. Backend Setup

I configured the Authentication and Authorization backend provided by my teammate and successfully executed the FastAPI server using Uvicorn.

Tasks performed

- Configured the backend project.
- Started the FastAPI server.
- Verified that the server was running successfully.
- Accessed the Swagger documentation.

```
C:\Users\DIKSHA\Downloads\backend>python -m uvicorn app.main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\DIKSHA\\Downloads\\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [12784] using WatchFiles
INFO: Started server process [34616]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:57880 - "GET / HTTP/1.1" 401 Unauthorized
INFO: 127.0.0.1:57880 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:61831 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:61831 - "GET /openapi.json HTTP/1.1" 200 OK
2026-08-08 00:02:11,311 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-08-08 00:02:11,317 INFO sqlalchemy.engine.Engine SELECT users.id AS use
rs_id, users.fullname AS users_fullname, users.email AS users_email, users.p
assword AS users_password, users.role AS users_role
FROM users
WHERE users.email = ?
LIMIT ? OFFSET ?
2026-08-08 00:02:11,317 INFO sqlalchemy.engine.Engine [generated in 0.00045s
] ('user@example.com', 1, 0)
2026-08-08 00:02:11,321 INFO sqlalchemy.engine.Engine ROLLBACK
INFO: 127.0.0.1:57149 - "POST /auth/register HTTP/1.1" 500 Internal Serv
er Error
ERROR: Exception in ASGI application
Traceback (most recent call last):
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-pac
kages\sqlalchemy\engine\base.py", line 1969, in _exec_single_context
    self.dialect.do_execute(
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-pac
kages\sqlalchemy\engine\default.py", line 952, in do_execute
    cursor.execute(statement, parameters)
sqlite3.OperationalError: no such table: users

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-pac
kages\uvicorn\protocols\http\httptools_impl.py", line 422, in run_asgi
    result = await app( # type: ignore[func-returns-value]
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-pac
```

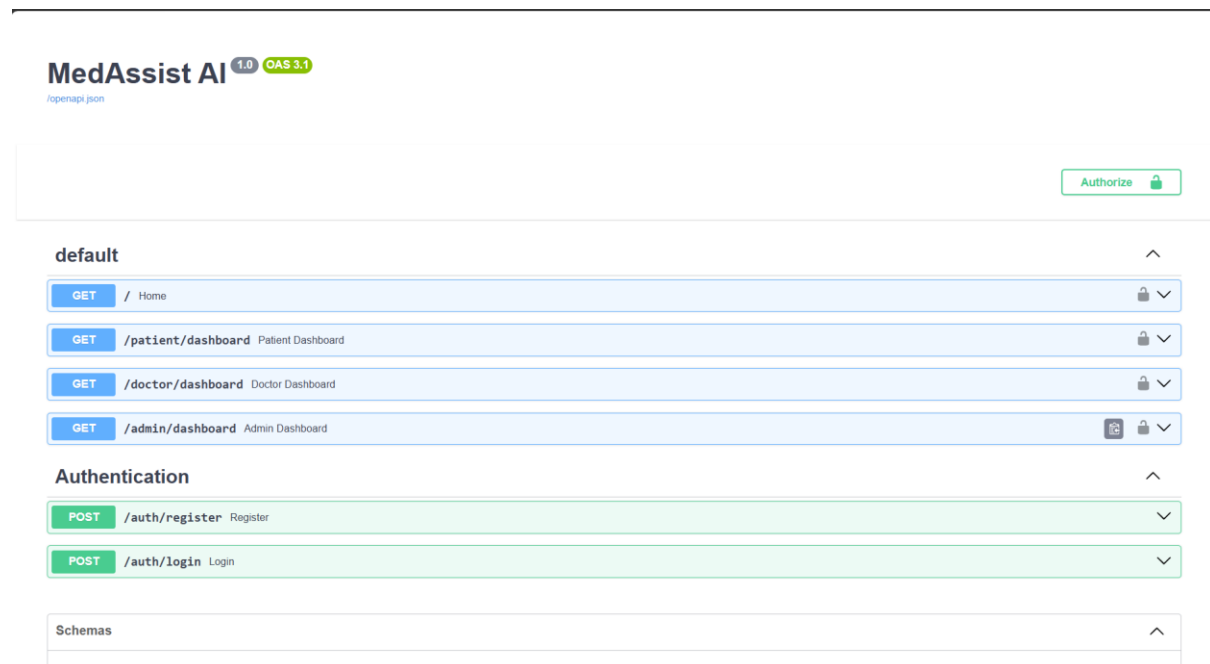
2. Swagger Documentation Review

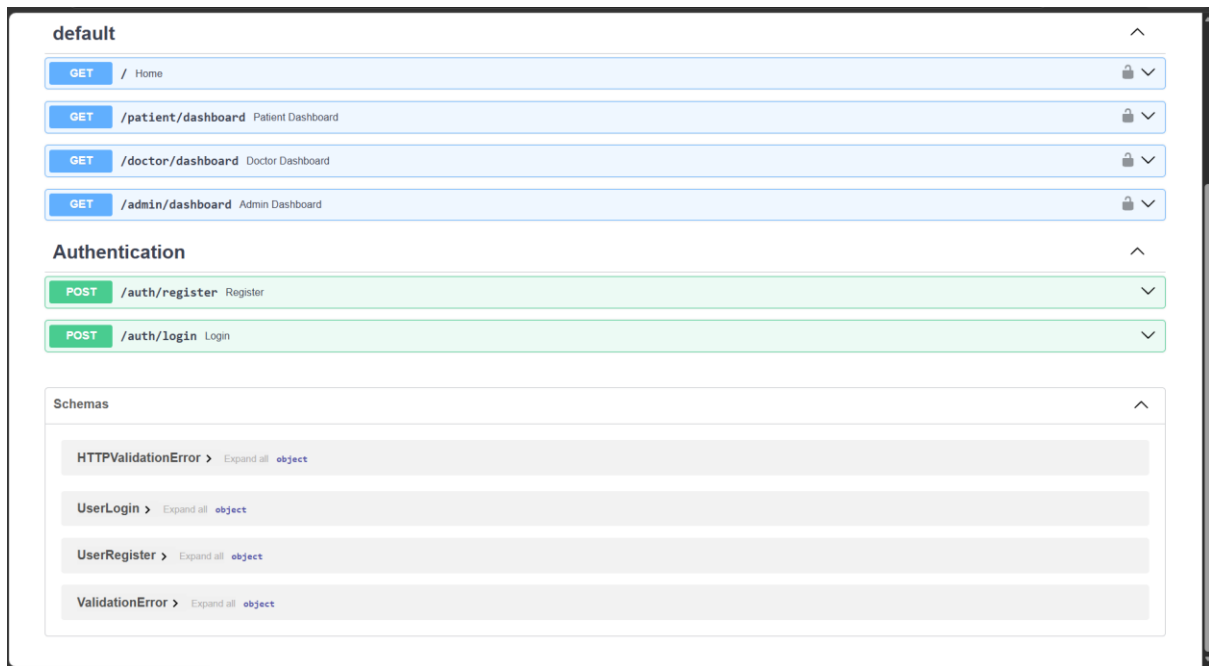
After running the backend, I explored the Swagger UI to understand the authentication flow.

The available endpoints included:

- Register
- Login
- Patient Dashboard
- Doctor Dashboard
- Admin Dashboard

This helped me understand the request structure that the frontend will eventually use during integration.





3. Authentication Module Testing

I attempted to test the Register and Login endpoints using the sample request bodies provided in Swagger.

During testing, both endpoints returned an Internal Server Error because the backend database had not been initialized.

The following issue was identified:

sqlite3.OperationalError: no such table: users

This indicates that the authentication logic exists but the database initialization has not yet been completed

Authentication

POST /auth/register Register

Parameters

No parameters

Request body required

application/json

Edit Value Schema

```
{
  "name": "string",
  "email": "user@example.com",
  "password": "string"
}
```

Execute

Clear

Responses

Prod

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/auth/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "string",
    "email": "user@example.com",
    "password": "string"
  }'
```

Request URL

http://127.0.0.1:8080/auth/register

Server response

Code Details

500 Error: Internal Server Error

Undocumented

Response body

```
Traceback (most recent call last):
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-packages\sqlalchemy\engine\base.py", line 1969, in _exec_single_context
    self.dialect.do_execute(
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-packages\sqlalchemy\engine\default.py", line 592, in do_execute
    cursor.execute(statement, parameters)
sqlalchemy.exc.OperationalError: no such table: users

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-packages\starlette\middleware\errors.py", line 164, in __call__
    await self.app(scope, receive, _send)
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-packages\starlette\middleware\exceptions.py", line 63, in __call__
    await wrap_app_handling_exceptions(self.app, conn)(scope, receive, send)
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-packages\starlette\_exception_handler.py", line 53, in wrapped_app
    raise exc
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-packages\starlette\_exception_handler.py", line 42, in wrapped_app
    await app(scope, receive, sender)
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-packages\fastapi\middleware\asyncexitstack.py", line 18, in __call__
    await self.app(scope, receive, send)
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-packages\starlette\routing.py", line 660, in __call__
    await self.middleware_stack(scope, receive, send)
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-packages\fastapi\routing.py", line 1883, in app
    await route.handle(scope, receive, send)
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-packages\fastapi\routing.py", line 1793, in handle
    await self.original_route.handle(scope, receive, send)
  File "C:\Users\DIKSHA\AppData\Local\Programs\Python\Python310\lib\site-packages\fastapi\routing.py", line 2738, in handle
```

Response headers

Response headers

content-length: 7353

content-type: text/plain; charset=utf-8

date: Fri, 07 Aug 2026 18:32:18 GMT

server: unicorn

Responses

Code	Description	Links
200	Successful Response	No links
Media type: application/json Controls Accept header. Example Value Schema "string"		
422	Validation Error	No links
Media type: application/json Example Value Schema { "detail": { { "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} } } }		

POST /auth/login Login

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

{
 "email": "user@example.com",
 "password": "string"
}

Execute

Clear

Responses

Curl

curl -X 'POST' \
 'http://127.0.0.1:8000/auth/login' \
 -H 'accept: application/json' \
 -H 'Content-Type: application/json' \
 -d '{
 "email": "user@example.com",

The screenshot displays a REST client interface with two panels. The top panel shows a 500 Internal Server Error response to a POST request at `http://127.0.0.1:8000/auth/login`. The response body contains a detailed Python traceback from SQLAlchemy, indicating a `sqlalchemy.exc.OperationalError` with the message "no such table: users". The error occurs in the `execute` method of the `ORMSession` class. The response headers show a content length of 7355, content type of `text/plain; charset=utf-8`, and a date of `Fri, 07 Aug 2020 18:33:32 GMT` from the `uvicorn` server.

The bottom panel shows a 200 Successful Response. The media type is set to `application/json`. The response body is a JSON object representing a validation error:

```
{
  "detail": [
    {
      "loc": [
        "string",
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

Below the response panels, there is a 'Schemas' section with a dropdown menu showing `HTTPValidationError` and an 'Expand all' button.

4. Analysis

Based on testing, I concluded that the backend authentication module still requires database configuration before it can be integrated with the frontend.

Therefore, frontend-backend integration was postponed until the backend implementation is completed.

Learning Outcome

During this milestone, I learned:

- How to configure and execute a FastAPI backend.
- How to use Swagger UI to inspect backend APIs.
- How Authentication and Authorization endpoints are structured.
- How to identify backend issues through server responses and error logs.
- The importance of verifying backend readiness before frontend integration.

Conclusion

During this milestone, I successfully verified and tested the Authentication and Authorization backend module provided by my teammate. The backend server and API documentation were successfully executed and examined. While testing, a database configuration issue was identified, preventing successful authentication. The issue was documented, and the frontend integration will proceed after the backend database is fully configured.